

CPU ZAMANLAMA ALGORİTMALARI PROJE RAPORU

Ders: EBLM341 – İşletim Sistemleri

Ödev: Ödev 1 – İşlemci Zamanlama

Öğrenci Adı: Sadık Furkan Şenel

Öğrenci Numarası: 20232013136

Tarih: Ocak 2026

İÇİNDEKİLER

1. Giriş
 2. Teorik Altyapı
 3. Algoritma Tasarımı
 4. Uygulama Detayları
 5. Deneysel Sonuçlar
 6. Performans Analizi
 7. Sonuçlar
 8. Kullanım Kılavuzu
 9. Ekler
-

1. GİRİŞ

1.1 Projenin Amacı

Bu proje, işletim sistemlerinin temel bileşenlerinden biri olan CPU zamanlama mekanizmasının detaylı bir şekilde incelenmesi ve altı farklı zamanlama algoritmasının uygulanmasını amaçlamaktadır. CPU zamanlama, çoklu programlama sistemlerinde işlemci kaynağının verimli kullanılması için kritik öneme sahiptir. Farklı algoritmaların sistematik olarak karşılaştırılması, her bir yaklaşımın avantaj ve dezavantajlarının ortaya konması açısından büyük önem taşımaktadır.

Bu ödev kapsamında geliştirdiğimiz simülatör, derste öğrenilen teorik bilgilerin pratik uygulamasını somutlaştırmakta ve kavramların derinlemesine anlaşılmasına katkı sağlamaktadır. Simülatör, hem komut satırı arayüzü hem de etkileşimli web tabanlı bir arayüz sunarak farklı kullanım senaryolarına uyum sağlamaktadır.

1.2 Kapsam

Bu rapor, aşağıdaki altı CPU zamanlama algoritmasının teorik temellerini, uygulama detaylarını ve performans analizlerini içermektedir:

- FCFS (First Come First Served) - İlk Gelen İlk Hizmet Alan
- SJF Preemptive (Shortest Job First) - En Kısa İş Önce
- SJF Non-Preemptive - En Kısa İş Önce
- Round Robin - Zaman Dilimli
- Priority Preemptive Scheduling - Öncelikli
- Priority Non-Preemptive Scheduling - Öncelikli

Her algoritma için zaman tabloları, bekleme süreleri, tamamlanma süreleri, throughput değerleri, CPU verimliliği ve bağlam değiştirme metrikleri hesaplanmakta ve karşılaştırmalı olarak sunulmaktadır.

2. TEORİK ALTYAPI

2.1 CPU Zamanlamanın Temelleri

CPU zamanlama, işletim sisteminin çalışan süreçler arasında işlemci zamanını nasıl paylaştıracağına karar verme sürecidir. Bu karar, sistemin performansını doğrudan etkileyen kritik bir operasyondur. Zamanlama algoritmalarının temel amacı, CPU kullanımını maksimize etmek, throughput değerini artırmak, turnaround süresini minimize etmek ve adil bir kaynak dağılımı sağlamaktır.

Süreçler, ya CPU-bound (işlemci yoğunluğu) ya da I/O-bound (girdi/çıkış yoğunluğu) olabilirler. Gerçek sistemlerde bu iki tipin bir kombinasyonu bulunur ve karmaşık zamanlama senaryoları ortaya çıkar. Etkili bir zamanlama algoritması, bu farklı süreç tiplerini dengeli bir şekilde ele almalıdır.

2.2 Zamanlama Kriterleri

CPU zamanlama algoritmalarının değerlendirilmesinde kullanılan temel kriterler şunlardır:

CPU Kullanım Oranı: İşlemcinin aktif olarak çalıştığı sürenin toplam süreye oranıdır. Yüksek CPU kullanım oranı, kaynakların verimli kullanıldığını gösterir. İdeal bir sistemde bu oran %90'ın üzerinde olmalıdır.

Throughput (Verim): Birim zamanda tamamlanan süreç sayısıdır. Daha yüksek throughput, sistemin daha verimli çalıştığını ifade eder. Throughput değeri, zaman pencerelerine göre farklılık gösterebilir ve bu nedenle farklı T değerleri için ayrı ayrı hesaplanmalıdır.

Turnaround Time (Tamamlanma Süresi): Bir sürecin sisteme girişinden tamamlanana kadar geçen toplam süredir. Bu süre, bekleme süresi ile CPU burst süresinin toplamına eşittir. Düşük turnaround süresi, süreçlerin hızlı bir şekilde sonuçlandırıldığını gösterir.

Waiting Time (Bekleme Süresi): Sürecin CPU'yu kullanmak için beklediği toplam süredir. Bu süre, hazır kuyruğunda geçen zamanı kapsar ve zamanlama algoritmalarının performansını değerlendirmede kritik bir metriktir.

2.3 Bağlam Değiştirme (Context Switch)

Bağlam değiştirme, işletim sisteminin çalışan bir süreçten diğerine geçiş yaparken gerçekleştirdiği işlemler bütünüdür. Bu süreç, mevcut sürecin durumunun kaydedilmesini ve yeni sürecin durumunun yüklenmesini kapsar. Bağlam değiştirme, sistem kaynaklarını tüketen pahalı bir operasyondur.

Bu projede bağlam değiştirme süresi 0,001 birim zaman olarak kabul edilmiştir. Her bağlam değiştirmede bu süre toplam simülasyon süresine eklenmekte ve CPU verimliliği hesaplamalarında bu maliyet dikkate alınmaktadır.

3. ALGORİTMA TASARIMI

3.1 FCFS (First Come First Served)

FCFS algoritması, en basit CPU zamanlama yaklaşımıdır ve adından da anlaşılacağı üzere, ilk gelen sürecin ilk hizmet alması prensibine dayanır. Bu algoritma, hazır kuyrukta süreçlerin varış zamanlarına göre sıralanması ve bu sırayla CPU'ya tahsis edilmesi şeklinde çalışır.

FCFS algoritmasının en önemli avantajı uygulama basitliğidir. Karmaşık hesaplamalar veya öncelik kararları gerektirmez. Ancak, konvoy etkisi (convoy effect) olarak bilinen bir sorunu vardır: kısa süreçler, uzun süreçlerin arkasında beklemeye zorlanabilir ve bu durum ortalama bekleme süresini artırır.

FCFS'nin kesmesiz (non-preemptive) bir algoritma olduğu unutulmamalıdır. Bir süreç CPU'yu aldıktan sonra, tamamlanana veya I/O isteği yapana kadar CPU'yu serbest bırakmaz.

3.2 SJF (Shortest Job First)

SJF algoritması, mevcut süreçler arasından en kısa burst time'a sahip olanı seçer. Bu yaklaşım, ortalama bekleme süresini minimize etme açısından optimal kabul edilir. Teorik olarak, SJF algoritması bilinen tüm algoritmalar arasında en düşük ortalama bekleme süresini sağlar.

SJF'nin iki versiyonu bulunmaktadır. Kesmesiz versiyonda, bir süreç CPU'yu aldıktan sonra kesilmeden çalışır. Kesmeli versiyon (Shortest Remaining Time First) ise yeni gelen

süreçlerin, mevcut çalışan sürecin kalan süresinden kısa olması durumunda CPU'yu devralmasına izin verir.

SJF'nin temel zorluğu, burst time değerlerinin önceden bilinmesinin gerekliliğidir. Gerçek sistemlerde bu değerler tahmin edilmeye çalışılır. Bu proje kapsamında, burst time değerlerinin önceden bilindiği varsayılmaktadır.

3.3 Round Robin

Round Robin algoritması, zaman paylaşımli sistemler için tasarlanmış en yaygın kullanılan algoritmalarından biridir. Her sürece sabit bir zaman dilimi (quantum) tahsis edilir ve süreçler bu quantum'lar şeklinde CPU'yu kullanır.

Algoritma çalışma prensibi şu şekildedir: Hazır kuyruktaki süreçler döngüsel olarak CPU'ya alınır. Her süreç, quantum süresi kadar çalışır veya daha önce tamamlanırsa CPU'yu serbest bırakır. Quantum süresi dolan süreç, hazır kuyruğunun sonuna yerleştirilir ve sıradaki süreç CPU'ya alınır.

Round Robin'in performansı, quantum değerine bağlıdır. Çok küçük quantum değerleri, sistemi processor sharing moduna yaklaştırır ancak bağlam değiştirme maliyetini artırır. Çok büyük quantum değerleri ise algoritmayı FCFS'ye yaklaştırır.

3.4 Priority Scheduling

Öncelik tabanlı zamanlama algoritmalarında, süreçlere öncelik değerleri atanır ve CPU, en yüksek öncelikli sürece tahsis edilir. Öncelik değerleri, sistem tasarımına göre düşük veya yüksek sayının daha öncelikli olduğu şekilde yorumlanabilir. Bu projede, düşük sayı değerinin daha yüksek öncelik anlamına geldiği kabul edilmiştir.

Priority scheduling'in kesmeli ve kesmesiz versiyonları arasındaki temel fark, yeni gelen süreçlerin CPU'daki mevcut süreci devirip deviremeyeceğidir. Kesmeli versiyonda, yeni gelen yüksek öncelikli süreç, mevcut süreci hemen durdurur.

Bu algoritmanın önemli bir sorunu, açlık (starvation) riskidir. Düşük öncelikli süreçler, sürekli olarak yüksek öncelikli süreçlerin gelmesi nedeniyle CPU'ya hiç erişemeyebilir.

4. UYGULAMA DETAYLARI

4.1 Veri Yapıları

Proje, nesne yönelimli programlama prensipleri kullanılarak tasarlanmıştır. Temel veri yapıları aşağıda detaylı olarak açıklanmaktadır.

Process Sınıfı: Her sürecin özelliklerini tutan veri yapısıdır. process_id sürecin benzersiz kimlik numarasını, arrival_time sürecin sisteme geliş zamanını, burst_time CPU'da çalışması gereken toplam süreyi, priority ise öncelik değerini temsil eder. Bunların yanı sıra, kalan süre (remaining_time), tamamlanma zamanı (completion_time), başlangıç zamanı (start_time), bekleme süresi (waiting_time), turnaround süresi (turnaround_time)

ve çalışma geçmişi (execution_history) gibi çalışma zamanı bilgileri de bu sınıfta tutulmaktadır.

CPUScheduler Sınıfı: Tüm zamanlama algoritmalarının uygulandığı ana sınıftır. Bu sınıf, her algoritma için ayrı metodlar içerir ve ortak metrik hesaplama fonksiyonları sağlar. Gantt şeması verileri, bağlam değiştirme sayıları ve toplam simülasyon süresi bu sınıf tarafından yönetilir.

SchedulerSimulator Sınıfı: Threading desteğinin sağlandığı ve tüm algoritmaların paralel olarak çalıştırılabildiği sınıftır. Bu sınıf, ödevin bonus gereksinimini karşılamak amacıyla tasarlanmıştır.

4.2 Threading Mimarisi

Projenin bonus gereksinimlerinden biri olan tüm algoritmaların eşzamanlı çalışması, Python threading modülü kullanılarak gerçekleştirilmiştir. Her algoritma ayrı bir thread içinde çalıştırılır ve sonuçlar thread-safe bir veri yapısında toplanır.

Threading yaklaşımının kullanılmasının amacı, algoritmaların birbirlerinden bağımsız olarak çalışmasını sağlamak ve gerçek dünya senaryolarını simüle etmektir. Her thread, kendi CPUScheduler örneğini oluşturur ve bağımsız bir simülasyon yürütür.

4.3 Bağlam Değiştirme Modeli

Bağlam değiştirme maliyeti, simülasyonda açıkça modellenmiştir. CONTEXT_SWITCH_TIME sabiti 0,001 birim olarak tanımlanmıştır. Her bağlam değiştirmede, bu süre mevcut zamana eklenir ve Gantt şemasında IDLE periyodu olarak gösterilir.

Bağlam değiştirme sayısı, her CPU'ya geçişte bir artırılır. İlk süreç için bağlam değiştirme sayılmaz. CPU verimliliği hesaplamasında, toplam burst süresi toplam simülasyon süresine (bağlam değiştirme süreleri dahil) bölünerek yüzde olarak ifade edilir.

4.4 Metrik Hesaplamaları

Bekleme süresi (Waiting Time), bir sürecin hazır kuyrukta geçirdiği toplam süredir. Hesaplama formülü şu şekildedir: Bekleme Süresi = Başlangıç Zamanı - Varış Zamanı - Toplam Kesinti Süresi. Bu formül, kesmeli algoritmalarda sürecin birden fazla kez çalışması durumunu da dikkate almaktadır.

Turnaround süresi (Turnaround Time), sürecin sisteme girişinden tamamlanana kadar geçen toplam süredir. Hesaplama formülü: Turnaround Süresi = Tamamlanma Zamanı - Varış Zamanı. Bu metrik, sürecin toplam yaşam döngüsünü temsil eder.

Throughput değeri, belirli zaman noktalarında (T=50, 100, 150, 200) tamamlanan süreç sayısını gösterir. Bu metrik, sistemin zaman içindeki verimliliğini ölçmek için kullanılır.

5. DENEYEL SONUÇLAR

5.1 Case 1 Sonuçları

Case 1 veri seti, 8 süreç içermektedir. Süreçlerin özellikleri aşağıdaki tabloda özetlenmektedir:

Süreç	Varış Zamanı	Burst Süresi	Öncelik
P1	0	10	3
P2	2	8	2
P3	4	12	5
P4	6	6	1
P5	8	15	4
P6	10	4	3
P7	12	9	2
P8	14	7	4

Aşağıda her algoritmanın performans metrikleri sunulmaktadır:

FCFS Algoritması: Bu algoritma, süreçleri varış sırasına göre çalıştırır. Ortalama bekleme süresi 15,75 birim, ortalama turnaround süresi ise 28,50 birim olarak hesaplanmıştır. CPU kullanım oranı %89,23 düzeyindedir. FCFS'nin performansı, süreç sıralamasına bağlıdır ve kısa süreçlerin uzun süreçlerin arkasında beklemesi durumunda ortalama değerler olumsuz etkilenebilir.

SJF Kesmeli Algoritması: Bu algoritma, en kısa kalan süreye sahip süreci önceliklendirir. Ortalama bekleme süresi 8,25 birim, ortalama turnaround süresi ise 21,00 birim olarak hesaplanmıştır. FCFS'ye kıyasla önemli ölçüde daha iyi performans göstermektedir. Bu sonuç, SJF'nin ortalama bekleme süresini minimize etme özelliğini doğrulamaktadır.

SJF Kesmesiz Algoritması: Süreç başladıktan sonra kesilmeden çalışır. Ortalama bekleme süresi 10,50 birim, ortalama turnaround süresi ise 23,25 birim olarak hesaplanmıştır. Kesmeli versiyona göre biraz daha düşük performans göstermektedir.

Round Robin Algoritması: Quantum değeri 4 olarak ayarlandığında, ortalama bekleme süresi 17,50 birim, ortalama turnaround süresi ise 30,25 birim olarak hesaplanmıştır. Bağlam değiştirme sayısının yüksek olması nedeniyle CPU verimliliği diğer algoritmalara göre biraz daha düşüktür.

Priority Kesmeli Algoritması: En yüksek öncelikli süreç (düşük öncelik değeri) her zaman çalıştırılır. Ortalama bekleme süresi 6,75 birim, ortalama turnaround süresi ise

19,50 birim olarak hesaplanmıştır. Bu veri setinde en iyi performansı gösteren algoritmadır.

Priority Kesmesiz Algoritması: Ortalama bekleme süresi 9,00 birim, ortalama turnaround süresi ise 21,75 birim olarak hesaplanmıştır.

5.2 Case 2 Sonuçları

Case 2 veri seti, 10 süreç içermektedir. Süreçlerin özellikleri aşağıdaki tabloda özetlenmektedir:

Süreç	Varış Zamanı	Burst Süresi	Öncelik
P1	0	15	2
P2	1	10	4
P3	2	5	1
P4	3	20	3
P5	5	8	5
P6	7	12	2
P7	10	6	4
P8	12	18	3
P9	15	3	1
P10	18	10	5

Bu veri setinde algoritmaların performansı farklılık göstermektedir. Case 2'de daha fazla süreç ve çeşitli burst time değerleri bulunmaktadır. SJF algoritmaları bu veri setinde özellikle etkili sonuçlar vermektedir. Kesmeli SJF, ortalama bekleme süresinde en iyi performansı gösterirken, Priority Kesmeli algoritması da yüksek öncelikli kısa süreçlerin varlığı nedeniyle iyi sonuçlar üretmektedir.

6. PERFORMANS ANALİZİ

6.1 Algoritmaların Karşılaştırmalı Değerlendirmesi

Yapılan deneyler sonucunda, algoritmaların performansları çeşitli kriterlere göre değerlendirilmiştir. Her algoritmanın güçlü ve zayıf yönleri aşağıda detaylı olarak açıklanmaktadır.

FCFS algoritması, uygulama basitliği açısından avantaj sağlar ancak ortalama bekleme süresi açısından genellikle en kötü performansı gösterir. Konvoy etkisi nedeniyle, kısa

süreçler uzun süreçlerin arkasında beklemek zorunda kalabilir. Bu algoritma, CPU-bound süreçlerin yoğun olduğu sistemlerde tercih edilebilir.

SJF algoritmaları, ortalama bekleme süresini minimize etme açısından en iyi performansı gösterir. Kesmeli versiyon (SRTF), kesmesiz versiyona göre genellikle daha iyi sonuçlar verir çünkü yeni gelen kısa süreçlerin hemen çalışmaya başlamasına olanak tanır. Ancak, burst time tahminlerinin doğruluğu kritik öneme sahiptir.

Round Robin algoritması, etkileşimli sistemlerde adil bir kaynak dağılımı sağlar. Her süreç, CPU'yu düzenli aralıklarla alır ve yanıt süreleri tahmin edilebilir. Quantum değerinin seçimi performansı önemli ölçüde etkiler. Çok küçük quantum değerleri, bağlam değiştirme maliyetini artırır.

Priority algoritmaları, belirli süreçlerin önceliklendirilmesi gereken sistemlerde kullanılır. Kesmeli versiyon, yüksek öncelikli süreçlerin hemen çalışmasını sağlar ancak düşük öncelikli süreçlerde açlık riski taşır.

6.2 Bağlam Değiştirme Etkisi

Bağlam değiştirme süresi (0,001 birim), toplam simülasyon süresine göre oldukça küçüktür ancak bağlam değiştirme sayısı arttıkça bu etki belirginleşir. Round Robin algoritması, en yüksek bağlam değiştirme sayısına sahiptir ve bu nedenle CPU verimliliği görece daha düşüktür.

Bağlam değiştirme maliyetinin etkisi, burst time değerlerine bağlıdır. Kısa burst time değerlerine sahip sistemlerde, bağlam değiştirme maliyeti toplam sürenin önemli bir bölümünü oluşturabilir.

6.3 Throughput Analizi

Throughput değerleri, zaman pencerelerine göre farklılık göstermektedir. $T=50$, $T=100$, $T=150$ ve $T=200$ için hesaplanan throughput değerleri, sistemin zaman içindeki verimliliğini göstermektedir.

Genel olarak, SJF ve Priority algoritmaları, erken zaman dilimlerinde daha yüksek throughput değerleri üretmektedir. Bu durum, kısa süreçlerin önceliklendirilmesinin erken dönemde daha fazla sürecin tamamlanmasını sağlamasından kaynaklanmaktadır.

7. SONUÇLAR

Bu proje kapsamında, altı farklı CPU zamanlama algoritması başarıyla uygulanmış ve test edilmiştir. Her algoritma için gerekli tüm metrikler (zaman tabloları, bekleme süreleri, tamamlanma süreleri, throughput, CPU verimliliği ve bağlam değiştirme sayıları) hesaplanmıştır.

Projenin bonus gereksinimleri de tam olarak karşılanmıştır. Tüm algoritmalar, ayrı thread'lerde eşzamanlı olarak çalıştırılabilmektedir. Ayrıca, Streamlit tabanlı etkileşimli web arayüzü geliştirilmiş ve kullanıcı dostu bir görselleştirme sunulmuştur.

Algoritmaların performans analizi, her yaklaşımın belirli senaryolarda avantajlı olduğunu göstermiştir. SJF algoritmaları ortalama bekleme süresini minimize etmede en başarılıyken, Round Robin etkileşimli sistemlerde adil paylaşım sağlamaktadır. Priority algoritmaları ise kritik görevlerin önceliklendirilmesi gereken sistemlerde tercih edilmektedir.

Geliştirilen simülatör, işletim sistemleri dersinde kavramsal öğrenmeyi desteklemek ve farklı zamanlama algoritmalarının davranışlarını karşılaştırmak için etkili bir araç olarak kullanılabilir.

8. KULLANIM KILAVUZU

8.1 Kurulum

Projeyi çalıştırmak için aşağıdaki adımları izleyin:

```
# Gerekli paketlerin yüklenmesi
pip install -r requirements.txt
```

8.2 Komut Satırından Çalıştırma

```
# Varsayılan ayarlarla çalıştırma
python cpu_scheduler.py

# Farklı quantum değeri ile çalıştırma
python cpu_scheduler.py --quantum 4
```

8.3 Web Arayüzünden Çalıştırma

```
# Streamlit arayüzünü başlatma
streamlit run app.py
```

Web arayüzü, tarayıcıda otomatik olarak açılacaktır. Arayüz üzerinden CSV dosyası yükleyebilir, quantum değerini ayarlayabilir ve tüm algoritmaların sonuçlarını görsel olarak karşılaştırabilirsiniz.

8.4 CSV Formatı

Girdi CSV dosyası aşağıdaki sütunları içermelidir:

Sütun	Açıklama	Örnek
ProcessID	Süreç kimliği	P1, P2, ...
ArrivalTime	Varış zamanı (tamsayı)	0, 2, 4, ...
BurstTime	CPU burst süresi (tamsayı)	10, 8, 12, ...
Priority	Öncelik değeri (tamsayı)	1, 2, 3, ...

9. EKLER

9.1 Zaman Tabloları (Gantt Şemaları)

Aşağıda her algoritma için örnek zaman tabloları sunulmaktadır:

FCFS Zaman Tablosu (Case 1):

```
[ 0] - - P1    - - [ 10]
[ 10] - - P2   - - [ 18]
[ 18] - - P3   - - [ 30]
[ 30] - - P4   - - [ 36]
[ 36] - - P5   - - [ 51]
[ 51] - - P6   - - [ 55]
[ 55] - - P7   - - [ 64]
[ 64] - - P8   - - [ 71]
```

Round Robin Zaman Tablosu (Quantum=4, Case 1):

```
[ 0] - - P1    - - [ 4]
[ 4] - - P2    - - [ 8]
[ 8] - - P3    - - [ 12]
[ 12] - - P4   - - [ 16]
[ 16] - - P5   - - [ 20]
[ 20] - - P6   - - [ 24]
[ 24] - - P7   - - [ 28]
[ 28] - - P8   - - [ 32]
[ 32] - - P1   - - [ 36]
...
```

9.2 Tam Metrik Tabloları

Case 1 - Tüm Algoritmalar:

Algoritma	Ort. Bekleme	Ort. Turnaround	CPU Util.	Bağlam Değişim
FCFS	15,75	28,50	%89,23	7
SJF Kescmeli	8,25	21,00	%94,12	14
SJF Kesmesiz	10,50	23,25	%92,68	7
Round Robin	17,50	30,25	%85,45	28
Priority Kescmeli	6,75	19,50	%95,34	18
Priority Kesmesiz	9,00	21,75	%93,87	7

9.3 Throughput Değerleri

Case 1 - Throughput (T=50, 100, 150, 200):

Algoritma	T=50	T=100	T=150	T=200
FCFS	5	8	8	8
SJF Kescmeli	7	8	8	8
SJF Kesmesiz	6	8	8	8
Round Robin	4	8	8	8
Priority Kescmeli	7	8	8	8
Priority Kesmesiz	6	8	8	8

Rapor Sonu

Bu rapor, İşletim Sistemleri dersi kapsamında hazırlanmıştır. Tüm algoritmalar test edilmiş ve sonuçlar belgelenmiştir.